Implement a solution for a Constraint Satisfaction Problem using Branch and Bound and Backtracking for n-queens problem or a graph coloring problem.

```
def solve_n_queens(n):
    board = [["." for _ in range(n)] for _ in range(n)]
    result = []

    # Function to check whether queen placement is safe
    def is_safe(row, col):

        # Check same column
        for i in range(row):
            if board[i][col] == "Q":
                return False

        # Check upper-left diagonal
        i, j = row - 1, col - 1
        while i >= 0 and j >= 0:
            if board[i][j] == "Q":
                return False
            i -= 1
            j -= 1

        # Check upper-right diagonal
        i, j = row - 1, col + 1
        while i >= 0 and j < n:
            if board[i][j] == "Q":
                return False
            i -= 1
            j += 1
```

```
    return True

# Backtracking function
def backtrack(row):

    # If all queens are placed
    if row == n:
        result.append(["".join(r) for r in board])
        return

    # Try placing queen in every column
    for col in range(n):

        # Branch and Bound condition
        if is_safe(row, col):

            # Place queen
            board[row][col] = "Q"

            # Recur for next row
            backtrack(row + 1)

            # Backtrack: remove queen
            board[row][col] = "."

# Start from first row
backtrack(0)

return result
```

```
# Example usage
n = 4
solutions = solve_n_queens(n)

print("Total solutions for", n, "-Queens:", len(solutions))

for sol in solutions:
    for row in sol:
        print(row)
    print()
```

Short Explanation for Oral:

N-Queens is a Constraint Satisfaction Problem where N queens must be placed on an N×N chessboard so that no two queens attack each other.

This program uses:

1. Backtracking – tries possible queen positions row by row and removes invalid choices.
2. Branch and Bound – unsafe positions are rejected immediately using is_safe() to reduce unnecessary searching.

Algorithm Steps:

1. Start placing queens from row 0.
2. Check whether the position is safe.
3. If safe, place the queen and move to the next row.
4. If no valid position exists, backtrack and try another column.
5. Repeat until all solutions are found.

Time Complexity:
Worst Case: O(N!)
Space Complexity: $O(N^2)$ for board storage.